Paralelismo funcional

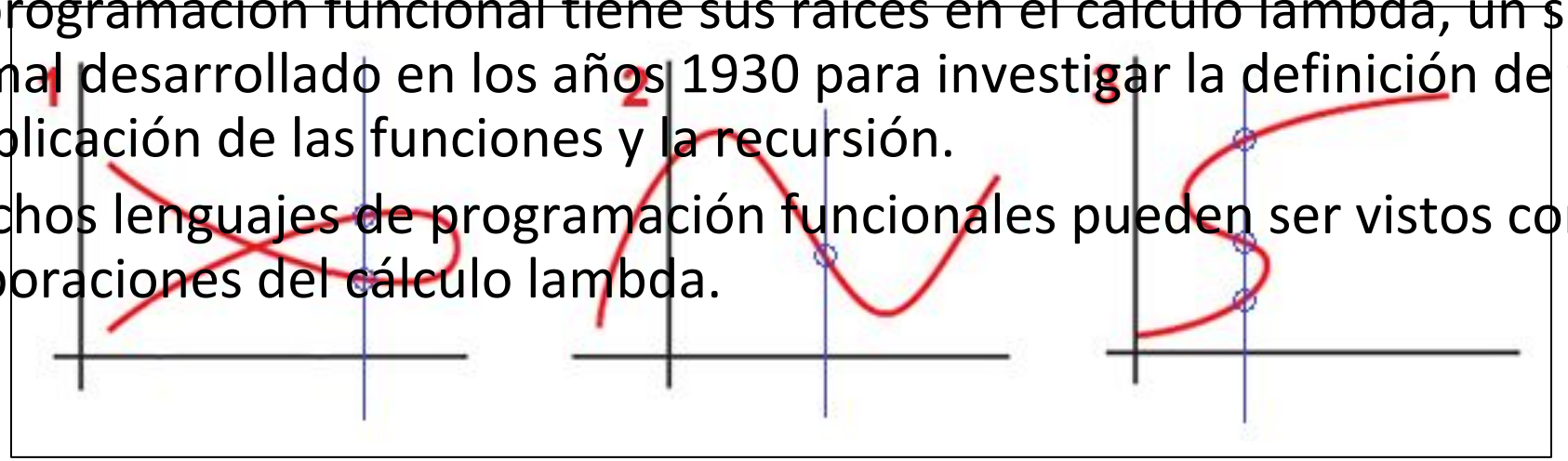
Octubre de 2026

Contenido

- Programación funcional
- Tareas futuras
- Memoización
- Streams de Java
- Determinismo y condiciones de carrera

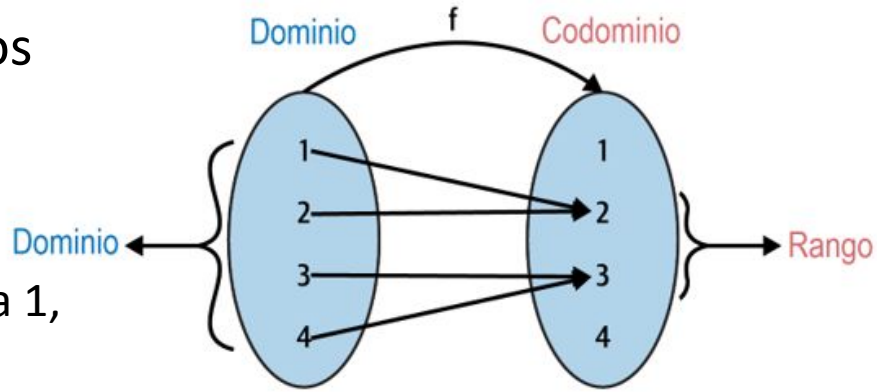
Programación funcional

- En ciencias de la computación, la programación funcional es un paradigma de programación declarativa ***basado en el uso de funciones matemáticas***, en contraste con la programación imperativa, que enfatiza los cambios de estado mediante la mutación/cambio de variables.
  - La programación funcional tiene sus raíces en el cálculo lambda, un sistema formal desarrollado en los años 1930 para investigar la definición de función, la aplicación de las funciones y la recursión.
  - Muchos lenguajes de programación funcionales pueden ser vistos como elaboraciones del cálculo lambda.



Funciones

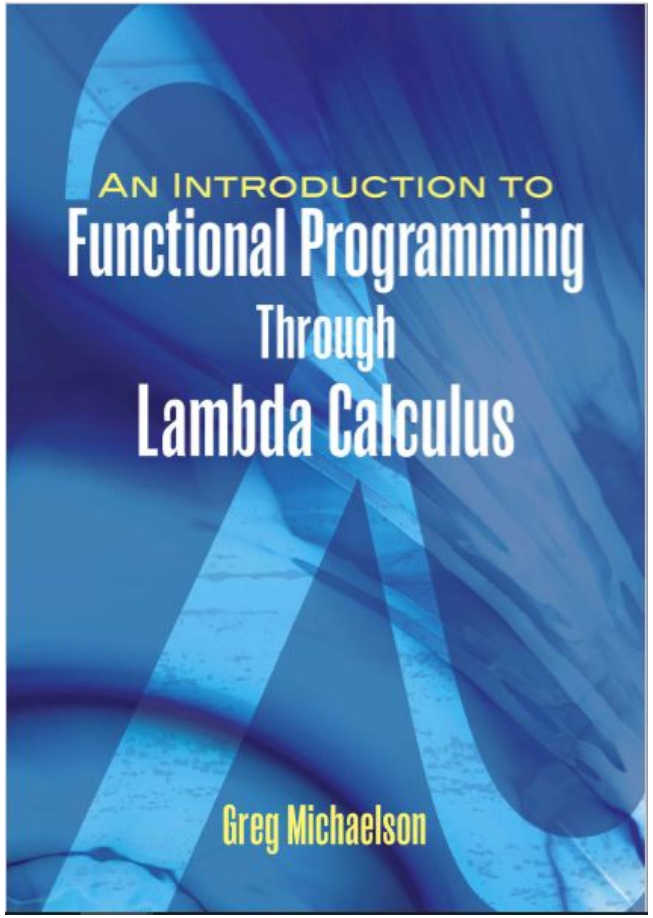
- En lenguaje llano, una función es una regla/norma que asocia cada elemento de un conjunto con algún elemento de otro conjunto.
- La idea es que tenemos un conjunto de cosas y asociamos cada una de ellas con algo de un segundo conjunto de cosas. Un ejemplo sencillo de matemáticas es la función elevar al cuadrado definida para todos los números enteros.
  - Esta función asigna cada número entero a su cuadrado: 1 va a 1, 2 va a 4, 3 va a 9, etc. Además, -1 va a 1, -2 va a 4, etc.
- Vemos, pues, que una función puede llevar dos valores distintos al mismo valor. Lo contrario no es cierto.
  - No se puede asignar un elemento a dos elementos diferentes.
- Las funciones se definen entre conjuntos.



Dominio y rango de una función

Cálculo lambda

- La programación funcional tiene sus raíces en el cálculo lambda, que constituye el lenguaje de programación más pequeño del mundo.
- Este reconocido texto ofrece una introducción accesible a los conceptos y técnicas de la programación funcional para estudiantes de matemáticas y ciencias de la computación.
- La exposición es lo menos técnica posible y no supone conocimientos previos ni de matemáticas ni de programación funcional.
- Ejemplos claros ilustran las ideas centrales, y a lo largo del texto aparecen numerosos ejercicios que refuerzan los conceptos clave.
    - Todos los problemas incluyen soluciones completas.



https://www.macs.hw.ac.uk/~greg/books/gjm.lambook88.pdf

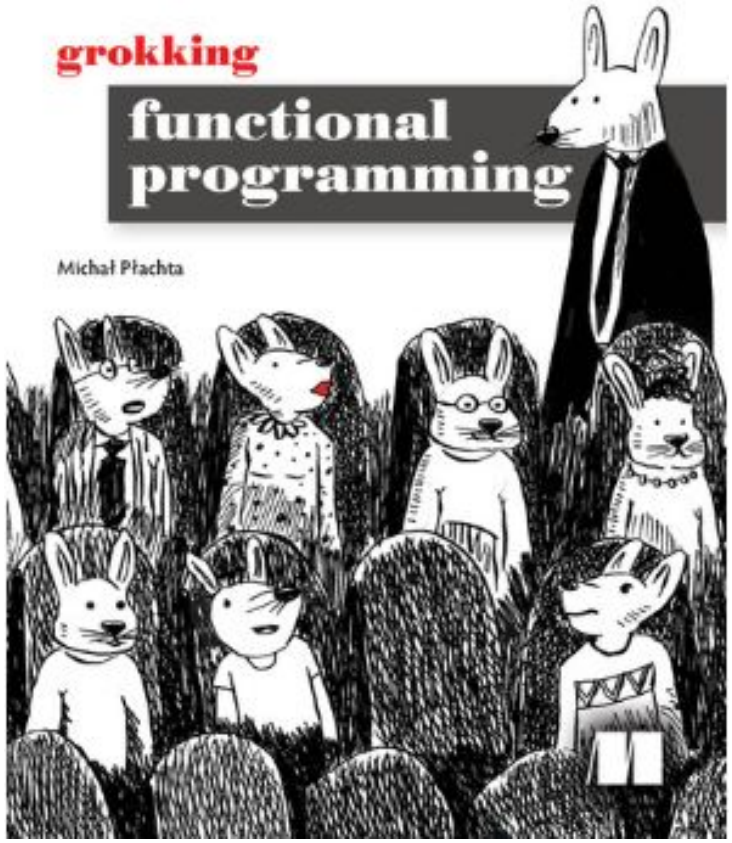
Programación funcional

- Los lenguajes de programación funcional, especialmente los puramente funcionales, han sido enfatizados en el ambiente académico y no tanto en el desarrollo comercial o industrial. Sin embargo, Scheme, Erlang, OCaml, Scala, F# y Haskell, han sido utilizados en aplicaciones comerciales e industriales por muchas organizaciones.
- La programación funcional también puede ser desarrollada en lenguajes que no están diseñados específicamente para la programación funcional. Perl, JavaScript, Python y Java (a partir de su versión 8), incorporan características de programación funcional.
- ***La programación funcional facilita el paso a programación paralela***

Funciones puras

- Las funciones (o expresiones) puramente funcionales NO tienen efectos secundarios (ni en memoria ni en operaciones de E/S):
  - Si no se utiliza el resultado de una expresión pura, se puede eliminar sin afectar a otras expresiones.
  - Si una función pura se llama con parámetros que no causan efectos secundarios, el resultado es constante con respecto a la lista de parámetros. Es decir, si la función pura se invoca de nuevo *con los mismos parámetros*, el mismo resultado será devuelto.
  - Si *NO hay dependencia* de datos entre dos expresiones puras, el orden de ejecución puede ser invertido, o *puede hacerse en paralelo*.
  - Si el lenguaje de programación no permite efectos secundarios, cualquier estrategia de evaluación se puede utilizar, lo que da la libertad al compilador para reordenar o combinar la evaluación de expresiones en un programa.

Un libro recomendado (no es el único)



- Dirigido a lectores con al menos un año de experiencia profesional desarrollando software en un lenguaje orientado a objetos como Java, y aunque usa Scala para los ejemplos, no exige conocimientos previos.
- Su contenido se organiza en tres partes: primero presenta las bases de la programación funcional, como funciones puras, inmutabilidad y la relación entre ambas; luego muestra cómo aplicar esos principios para resolver problemas reales, incluyendo manejo de errores, diseño funcional, interacción segura con efectos externos, streams y concurrencia; y finalmente desarrolla una aplicación funcional completa usando Wikidata, para integrar todo lo aprendido y mostrar también cómo probar y mantener el sistema ante cambios importantes.
- El libro incluye ejemplos de código, explicados y anotados, con un formato pensado para resaltar los conceptos y facilitar el aprendizaje práctico.

Tareas futuras

De las funciones puras al paralelismo

- Por ejemplo, si se escribe un programa como una función pura, por ejemplo

  `A = f(B)` [*El valor de A solo depende de B*]

  NOTA: Si se invoca muchas veces f(B) con el mismo valor B, SIEMPRE se tendrá el mismo resultado A.

- Y luego se utiliza el resultado retornado por f(B) para calcular otras dos funciones

  `C = g(A)` [*El valor de C solo depende de A*]
  `D = h(A)` [*El valor de D solo depende de A*]

Después de tener el valor de ***A***, el cálculo de ***C*** y de ***D*** se puede hacer en cualquier orden, incluso se puede hacer en PARALELO.

¿Cómo expresar el paralelismo funcional? Con **'futuros'**

El *pager* en la zona de comidas





"Placeholder"
[referente]



Objeto prometido
Objeto futuro

Tareas futuras (future task)

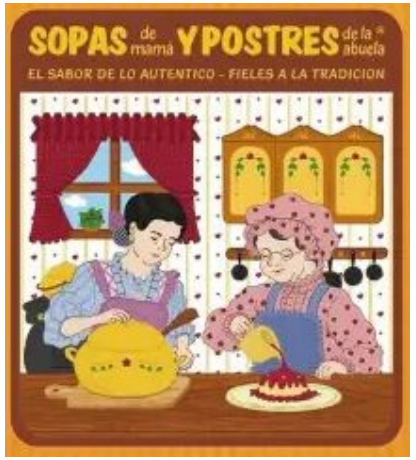
- El concepto de tarea asincrónica se puede extender al concepto de tareas futuras (*future task*), asociadas con objetos prometidos (*promise objects*) [El futuro es la vista de lectura del resultado; la promesa es el contenedor que el productor completa una sola vez]. Las tareas futuras son tareas con valores de retorno, y un objeto futuro es un "manejador" para acceder a un valor retornado por una tarea [en un tiempo posterior].
- El concepto de 'futuros' fue propuesto por Baker y Hewitt (1977) y Multilisp (Halstead, 1985) lo llevó a la programación paralela.
- En Multilisp se presenta un llamado de función **`(future E)`** [en sintaxis Lisp], donde **E** es cualquier expresión. Este llamado de función hace dos cosas:
  1. Inmediatamente retorna un *placeholder* para el resultado de E.
  2. Inicia, de forma concurrente, la evaluación de la expression E.

Tareas futuras (future task)

kokoriko

QBANO Delicioso!

EL CORRAL EST. 1983 LA RECETA ORIGINAL





Los locales están trabajando en paralelo para atender los cuatro pedidos

Tareas futuras (future task)

- Visitar
  - https://en.wikipedia.org/wiki/Futures_and_promises



Tareas futuras (future task)



- Hay dos operaciones claves que pueden ser realizadas sobre un objeto futuro ***A***:
  - ***Asignación (Assignment)*** — El objeto ***A*** puede ser asignado a una referencia para un objeto futuro retornado por una tarea de la forma:

    futuro { ⟨ tarea-con-valor-de-retorno ⟩ }

    - El contenido del objeto futuro está restringido a ser de *asignación única* (similar a una variable `final` en Java), y dicha asignación no puede ser modificada después que la tarea futura haya retornado la asignación.
  - ***Bloqueo de lectura (Blocking read)*** — la operación ***A.get()*** espera hasta que la tarea asociada con el objeto futuro **A** haya sido completada (con un valor o con una excepción), y entonces propaga el valor retornado por la tarea como el valor retornado por **A.get()**.
    - Cualquier sentencia ***S*** ejecutada después de la operación ***A.get()*** puede garantizar que la tarea asociada con el objeto futuro ***A*** debió completarse antes de que ***S*** inicie su ejecución.
- Estas operaciones están cuidadosamente definidas para evitar la posibilidad de que ocurra una *condición de carrera* para algún valor retornado por las tareas, razón por la cual los 'futuros' son adecuados para hacer paralelismo funcional.

Recordando Java: La palabra reservada `final`

- En una aplicación posiblemente nos encontremos con algún valor que permanece constante durante la ejecución. Podemos agregar a la definición de la variable el modificador `final`, que indica que a esa variable solo se le puede asignar un valor u objeto una única vez. La sintaxis es la siguiente:

```
final tipo_variable nombre_de_variable = valor;
```

- Ejemplo

```
final int unaConstante = 10;
```

- Si tratamos de modificar el valor de una variable con el modificador final, el compilador indicará un error. Una vez definido el valor no se puede modificar.
  - *Realmente, lo que permanece constante es la asignación de un objeto a esa variable, no los atributos internos que pueda tener ese objeto.*

https://es.wikibooks.org/wiki/Programaci%C3%B3n_en_Java/La_palabra_reservada_final

Recordando: Condición de carrera

- Condición de carrera o condición de secuencia (*race condition*) es un concepto usado en electrónica y en programación de computadores.
- Cuando la salida o estado de un proceso es dependiente de una secuencia de eventos que se ejecutan en orden arbitrario y van a trabajar sobre un mismo recurso compartido, se puede producir un error cuando no se ejecutan en el orden que el programador esperaba.
  - El término se origina por la similitud de dos procesos «compitiendo» en una carrera por llegar antes que el otro.
  - El estado y la salida del sistema dependerán de cuál «llegó» primero.
  - Pueden presentarse inconsistencias y comportamientos impredecibles y NO compatibles con un sistema determinista.



https://es.wikipedia.org/wiki/Condici%C3%B3n_de_carrera

Un futuro NO bloquea el programa; esperar su valor sí

A_{futuro} = futuro{f(B)}

C_{futuro} = futuro{g(A_{futuro}.get())}
D_{futuro} = futuro{h(A_{futuro}.get())}

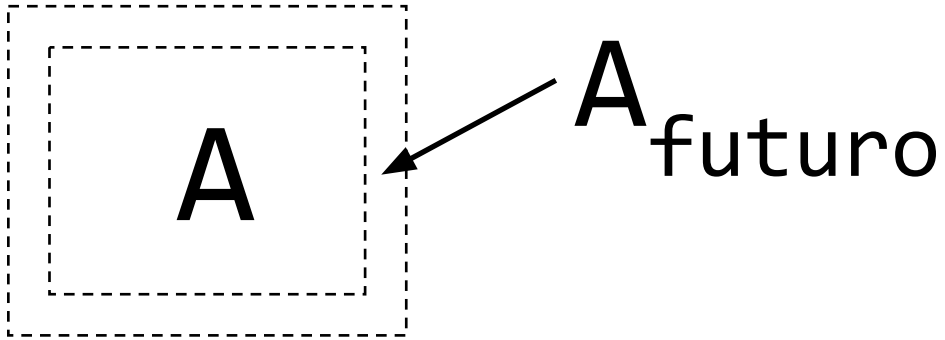
Al crear el futuro se entrega un *placeholder* del objeto prometido y la función empieza a ejecutarse de forma asíncrona (en paralelo, si hay hilos disponibles). El programa pasa a la siguiente instrucción: crear una tarea futura no bloquea.

- Cuando la tarea que está calculando, por ejemplo, C_{futuro} necesite el valor de A, ejecuta A_{futuro}.get(). En este momento la tarea se bloquea hasta que A esté disponible.
- Así se garantiza que g y h nunca usen A antes de que f termine. Si además f, g y h son funciones puras (sin estado compartido mutable), el programa no tiene condiciones de carrera.

Un “futuro” es un “manejador” del objeto

A_{futuro} = `futuro{f(B)}`

- La diferencia entre A_{futuro} y A es que A_{futuro} es una “envoltura” para A.



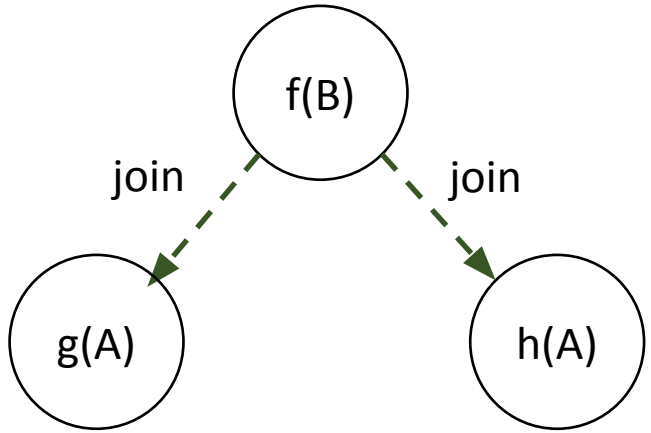
La “envoltura” es la *referencia al objeto* retornado por la tarea.

Object reference

- El término usual en Java es "*object reference*" antes que “*object handler*"
- Una referencia es un puntero opaco: identifica de manera única un objeto en la JVM, pero no permite aritmética ni acceso a la dirección.

Grafo computacional para tareas futuras

A_{futuro} = futuro{f(B)}
C_{futuro} = futuro{g(A_{futuro}.get())}
D_{futuro} = futuro{h(A_{futuro}.get())}



En los grafos computacionales estudiados antes, los enlaces tipo Join se presentaban al concluir una operación realizada en paralelo. Con las tareas futuras los enlaces tipo join se presentan cuando se realizan operaciones get()

Las tareas futuras permiten crear programas en paralelo

- Las tareas futuras permiten convertir programas funcionales en programas paralelos.
- Si otra parte del programa necesita consumir el resultado de la tarea futura, solo debe realizar la operación get().
- Las tareas futuras permiten crear una amplia gama de programas en paralelo y grafos computacionales que son más generales que los que se pueden hacer con async y finish (fork-join)

Tareas futuras en Java usando el framework Fork/Join

- Diferencias entre tareas futuras y tareas “normales” (las que mencionamos en la session anterior con FJ)
  - Una Tarea Futura extiende La clase RecursiveTask en el framework FJ, una tarea normal/regular extiende la clase RecursiveAction
  - El método compute() de una tarea futura debe retornar un tipo no vacío (no-void)
  - Cuando se invoca el método join() [por ejemplo left.join()] en ambos casos espera a que la tarea left termine, pero para las tareas futuras join() retorna un valor.
- Visitar https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/RecursiveTask.html

Future: el modelo conceptual

```
ExecutorService pool = Executors.newFixedThreadPool(4);
Future<Integer> aF = pool.submit(() -> f(b));
Future<Integer> cF = pool.submit(() -> g(aF.get()));
Future<Integer> dF = pool.submit(() -> h(aF.get()));
int r = cF.get() + dF.get();
```

El costo de esperar

Cada `get()` dentro de una tarea bloquea un hilo del pool. Si todos los hilos esperan resultados de tareas que no consiguen hilo, el programa se detiene.

	Future (Java 5)	CompletableFuture (Java 8)
Usar el resultado	Solo con get(), que bloquea	Encadenando continuaciones, sin bloquear
Completarlo manualmente	No: solo lo completa la tarea	Sí, con complete() (promesa)

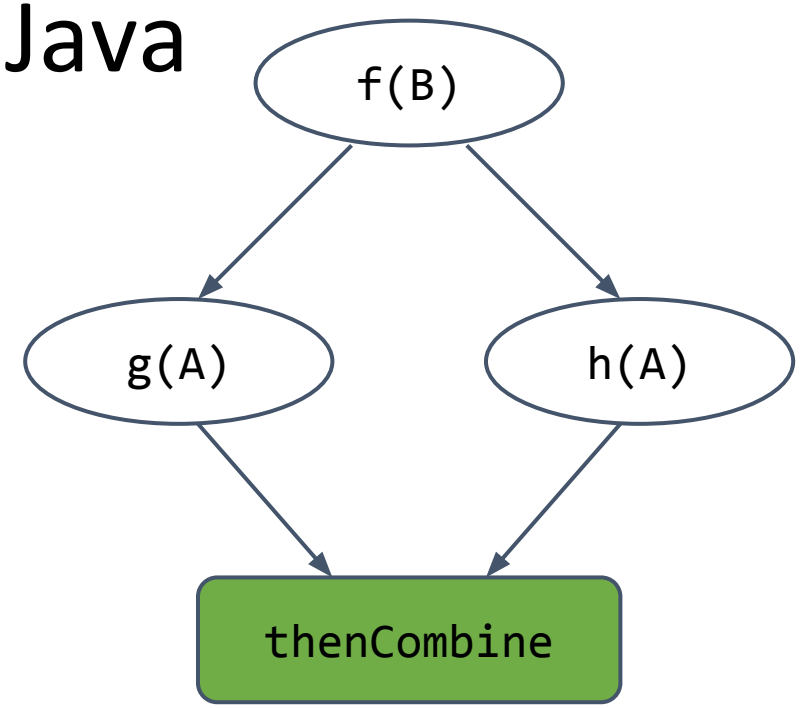
Future basta para el modelo conceptual. CompletableFuture evita bloquear hilos y permite componer el grafo.

Con Future podemos escribir exactamente el modelo que vimos en clase, pero cada get() dentro de una tarea deja un hilo bloqueado esperando, y si el grafo crece podemos quedarnos sin hilos libres; por eso Java ofrece `CompletableFuture`*, que veremos a continuación.*

CompletableFuture: los futuros en Java

```
var aF = CompletableFuture.supplyAsync(() -> f(b));
var cF = aF.thenApplyAsync(a -> g(a));
var dF = aF.thenApplyAsync(a -> h(a));
int r  = cF.thenCombine(dF, Integer::sum).join();
```

Notación de la clase	Java
futuro{f(B)}	supplyAsync(() -> f(b))
futuro{g(Afuturo.get())}	aF.thenApplyAsync(a -> g(a))
join del grafo	cF.thenCombine(dF, ...)
A.get()	join()



El mismo grafo de la lámina "Grafo computacional para tareas futuras"

`thenApplyAsync` no deja un hilo bloqueado esperando a A: registra lo que debe ejecutarse cuando A esté listo.

Este es el mismo grafo de futuros que ya conocemos, escrito en Java con `CompletableFuture`: en lugar de bloquear un hilo esperando a A, reg*i*stramos lo que debe pasar cuando A esté listo, y `thenCombine` hace el join al final.

CompletableFuture garantiza que cada paso se ejecute solo cuando su resultado previo está listo, sin bloquear hilos mientras espera.

Memoización

Binomio de Newton

- El binomio de Newton es un algoritmo que permite calcular una potencia cualquiera de un binomio, para ello se emplean los coeficientes binomiales, que no son más que una sucesión de números combinatorios. La fórmula general del binomio de Newton es:

$$(a+b)^n = \binom{n}{0} a^n + \binom{n}{1} a^{n-1}b + \binom{n}{2} a^{n-2}b^2 + \ldots + \binom{n}{n-1} ab^{n-1} + \binom{n}{n} b^n$$

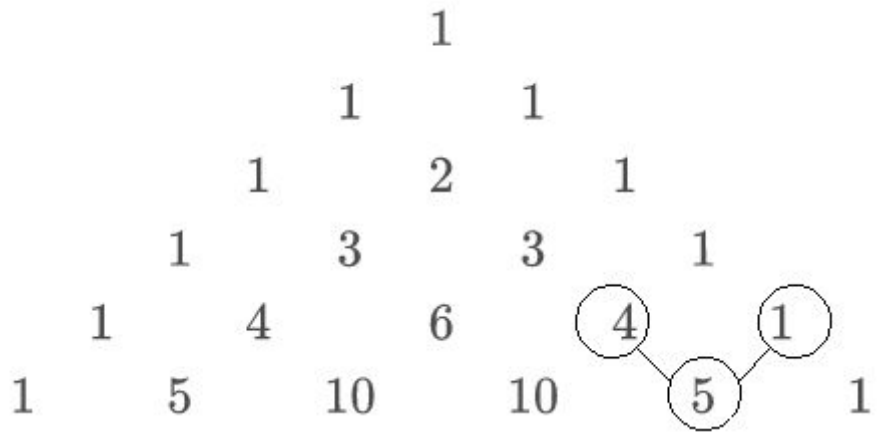
- Los números combinatorios que aparecen en la fórmula son precisamente los llamados coeficientes binomiales.
- Ejemplo

$$\begin{aligned}(a+b)^4 &= \binom{4}{0} a^4 + \binom{4}{1} a^3 b + \binom{4}{2} a^2 b^2 + \binom{4}{3} ab^3 + \binom{4}{4} b^4 \\ &= a^4 + 4a^3 b + 6a^2 b^2 + 4ab^3 + b^4\end{aligned}$$

https://www.sangakoo.com/es/temas/binomio-de-newton-y-triangulo-de-pascal

Triángulo de Pascal

- Pascal ideó una manera sencilla de calcular números combinatorios (aunque algunos la atribuyen a Tartaglia):



El método recibe el nombre de triángulo de Pascal y se construye de la siguiente forma (por filas y de arriba a abajo):

a. En el vértice se coloca un 1.
b. Cada fila empieza y acaba en 1.
c. Los otros números de la fila son siempre la suma de los dos que tiene justo encima. En la figura, 5 = 4 + 1

Triángulo de Pascal

- En la figura anterior, para la última fila mostrada, se construiría con los siguientes números combinatorios consecutivos:

$$\binom{5}{0}, \quad \binom{5}{1}, \quad \binom{5}{2}, \quad \binom{5}{3}, \quad \binom{5}{4}, \quad \binom{5}{5}$$

- El término general del desarrollo de $(a+b)^n$ es:

$$\binom{n}{k} a^{n-k} b^k$$

- Para calcular el vigésimo término de $(x + y)^{30}$ se tendría que es

¿Se debe siempre volver a calcular los números binomiales?

$$\binom{30}{19} x^{30-19} y^{19} = 54627300 x^{11} y^{19}$$

Memoización

- Dice Memoización, NO dice Memorización. Las palabras se parecen, pero son distintas, en la escritura y el concepto.
- En computación, la memoización o memoisación es una técnica de optimización utilizada para mejorar la rapidez de los programas para computador mediante el almacenamiento (en cache) de los resultados obtenidos de llamadas a funciones “costosas” y, cuando las mismas entradas ocurren otra vez, retornar el resultado guardado en cache evitando realizar el cálculo de la función “costosa” nuevamente.
- El término "memoización" fue acuñado por Donald Michie en 1968 y es una palabra derivada del Latín "memorandum" (“para ser recordado").
- El cálculo de los coeficientes binomiales se puede hacer utilizando memoización.

Memoización

- La memoización recuerda los resultados de los llamados a una función $f(x)$ de la siguiente forma:
  - Se crea una estructura de datos que almacene el conjunto

$$\{(x_1, y_1 = f(x_1)), (x_2, y_2 = f(x_2)), \ldots\}$$

  para cada llamada de $f(x_i)$ que retorna y_i
  - Luego buscar en dicha estructura de datos cuando se procesen las invocaciones de la forma f (x') teniendo que x' es igual a uno de los x_i de entrada para los cuales $f(x_i)$ fue calculada.

Memoización

- El patrón de memoización se presta para paralelizar mediante futuros.
- La estructura de datos de la memoización se modifica para que almacene $\{(x_1, y_1 = future(f(x_1))), (x_2, y_2 = future(f(x_2))), \ldots\}$.
- La búsqueda devuelve un futuro, y el valor se obtiene con `get()`. Si el cálculo aún está en curso, `get()` espera; si ya terminó, retorna de inmediato.
- Para que cada valor se calcule una sola vez, la inserción en la tabla debe ser atómica

Memoización con futuros: binomiales

```
record Par(int n, int k) {}
Map<Par, CompletableFuture<Long>> memo = new ConcurrentHashMap<>();

CompletableFuture<Long> binom(int n, int k) {
  if (k == 0 || k == n) return CompletableFuture.completedFuture(1L);
  var promesa = new CompletableFuture<Long>();
  var previa = memo.putIfAbsent(new Par(n, k), promesa);
  if (previa != null) return previa;   // calculado o en curso
  CompletableFuture.supplyAsync(() -> binom(n - 1, k - 1))
      .thenCompose(f -> f) // rama izquierda en otra tarea
      .thenCombine(binom(n - 1, k), Long::sum)
      .thenAccept(promesa::complete); // se cumple la promesa
  return promesa;
}
```

C(30, 19) = 54 627 300

109 254 599

llamadas sin memoización

209

entradas en la tabla con memoización

`putIfAbsent` es atómico: cada (n, k) se calcula una sola vez, aunque lo pidan varias tareas a la vez.

La tabla guarda trabajo terminado y trabajo en curso. Solo es correcto si la función es pura.

La tabla ya no guarda valores sino futuros: gracias a putIfAbsent, cada coeficiente se calcula una sola vez aunque varias tareas lo pidan al mismo tiempo, y así pasamos de más de 109 millones de llamadas a solo 209 cálculos, siempre que la función sea pura.

Streams de Java

Un Stream permite procesar los elementos de una colección de manera declarativa

- Para explicar qué es un Stream en Java, podemos utilizar una analogía con un granjero que necesita esquilar un rebaño de ovejas.
- Imaginemos una granja con un rebaño de diez ovejas, algunas blancas y otras negras.
- Tenemos tres elementos:
  1. El rebaño: una colección de objetos Oveja.
  2. Esquilar: una operación que se aplica a cada objeto del rebaño.
  3. El Stream: un mecanismo que permite procesar el rebaño mediante una secuencia de operaciones.



Un Stream NO es el rebaño NI la acción de esquilar. Es una abstracción para procesar los elementos [ovejas] provenientes del rebaño.

Programación imperativa: indicar cómo esquilar

```
List<Oveja> rebano = obtenerRebano();


for (Oveja oveja : rebano) {
esquilar(oveja);
}
```

En este caso le estamos indicando a Java:

1. Tome una oveja del rebaño.
2. Ejecute la operación esquilar.
3. Pase a la siguiente oveja.
4. Repita hasta terminar.

- Aquí controlamos explícitamente el recorrido, le decimos al Java cómo debe esquilar las ovejas.



Programación declarativa: indicar qué hacer

```
List<Oveja> rebano = obtenerRebano();
rebano.stream().forEach(this::esquilar);
```

En este caso

- `rebano`: colección de objetos `Oveja`.
- `stream()`: crea un *stream* de procesamiento.
- `forEach()`: aplica la función `esquilar()` a cada oveja.

Para cada oveja del rebaño, ejecute la operación de esquilar.

Ya no escribimos explícitamente el ciclo `for`. Le delegamos el recorrido a la API Stream.



Programación declarativa: indicar qué hacer

- **Podemos procesar solo las ovejas blancas**

```
rebano.stream()
.filter(Oveja::esBlanca)
.forEach(this::esquilar);
```



Operación	Significado en la analogía
`stream()`	Crear el Stream de procesamiento del rebaño
`filter()`	Seleccionar las ovejas blancas
`forEach()`	Aplicar la operación de esquilar

`filter()` es una operación intermedia, mientras que `forEach()` es una operación terminal que provoca el procesamiento del stream.

¿Y si queremos esquilar en paralelo?

```
List<Oveja> rebano = obtenerRebano();
rebano.parallelStream()
    .forEach(this::esquilar);
```

En el caso donde no se indica de manera explícita que se debe hacer la operación de manera paralela y la mostrada aquí, ***expresamos la misma operación sobre los mismos datos***. Lo que cambia es la estrategia de ejecución.
El stream paralelo permite distribuir el trabajo entre distintos hilos, generalmente utilizando el ForkJoinPool común.



Sin embargo, no garantiza que cada oveja se procese por un hilo diferente, ni que el procesamiento paralelo sea siempre más rápido.

Stream no significa necesariamente paralelismo

- Un Stream es una abstracción para expresar operaciones sobre un conjunto de elementos.
- El paralelismo es solamente una de sus posibles modalidades de ejecución.
- Además, un stream puede describir un pipeline con diferentes operaciones.

```
List<String> nombres =
    rebano.stream()

.filter(Oveja::necesitaEsquila)
          .map(Oveja::getNombre)
          .sorted()
          .toList();
```

`filter` selecciona las ovejas que necesitan esquila.
`map` transforma cada oveja seleccionada en su nombre.
`sorted` ordena los nombres.
`toList` produce una nueva lista.

Este pipeline no esquila las ovejas: solo obtiene sus nombres. Esta distinción permite explicar que los streams también pueden transformar, filtrar, agrupar y agregar datos, no solamente ejecutar acciones.

En resumen

- Un Stream en el lenguaje Java es una abstracción que permite describir declarativamente una secuencia de operaciones [pipeline] sobre elementos provenientes de una fuente de datos, sin programar explícitamente el cómo, y que puede ejecutarse de forma secuencial o paralela.

```
rebano.parallelStream().forEach(this::esquilar);
```

Rebaño = colección.
Esquilar = función.
Stream = abstracción de procesamiento.
Parallel = modalidad de ejecución.

La programación declarativa no elimina el algoritmo; permite expresar qué procesamiento se requiere y delegar parte de su ejecución a la librería. En este caso, Java se ocupa de recorrer y, si corresponde, repartir las tareas entre hilos.

Diferencias entre colección y Stream

Característica	Colección	Stream
Almacena elementos	Sí	No
Permite recorrer elementos	Sí	Sí
Permite filtrar y transformar	Mediante operaciones explícitas o auxiliares	Mediante operaciones declarativas
Puede procesar en paralelo	Mediante mecanismos adicionales	Sí, con `parallelStream()`
Puede reutilizarse	Generalmente sí	No
Puede modificar sus elementos	Depende de la colección	Puede realizar acciones con efectos, aunque se desaconseja modificar la fuente durante el procesamiento

Una colección es una estructura que almacena elementos; un Stream es una abstracción que permite describir declarativamente cómo procesarlos, sin necesidad de programar explícitamente su recorrido.

El rebaño es la colección, las ovejas son los elementos y el Stream organiza su procesamiento, mientras que `esquilar()` representa la operación aplicada a cada una de las ovejas.

¿Los Streams solamente funcionan con colecciones?

- No. También pueden obtener elementos de otras fuentes:

```
// Desde una colección
lista.stream();

// Desde un arreglo
Arrays.stream(arreglo);
```

```
// Desde valores individuales
Stream.of(1, 2, 3, 4, 5);

// Generación de valores
Stream.iterate(1, n -> n + 1);

// Desde un archivo de texto
Files.lines(Path.of("datos.txt"));
```

Stream Gatherers: nuevas formas de usar streams

- Java ha incorporado nuevas maneras de trabajar con Streams que van mucho más allá de elegir entre procesamiento secuencial y paralelo.
- Stream Gatherers se incorporó definitivamente en Java 24 (marzo de 2025).

Año	Versión de Java	Novedad
2014	Java 8	Stream, parallelStream, filter, map, reduce, forEach
2017	Java 9	takeWhile, dropWhile, ofNullable y API reactiva Flow
2021	Java 16	mapMulti() y toList()
2024, 19 de marzo	Java 22	Primera versión preliminar de Stream Gatherers (JEP 461)
2024, 17 de septiembre	Java 23	Segunda versión preliminar (JEP 473)
2025, 18 de marzo	Java 24	Versión definitiva de Stream Gatherers (JEP 485)

Stream Gatherers con el rebaño

- Ahora imaginemos que el granjero necesita resolver problemas nuevos:
  1. Organizar el rebaño en grupos de cinco.
  2. Tener exactamente tres esquiladores trabajando concurrentemente.
  3. Saber cuánta lana se ha obtenido de cada oveja.
  4. Examinar grupos consecutivos de tres ovejas.
  5. Obtener un resumen del peso total de la lana.











`windowFixed()` — organizar ovejas en lotes

```
rebano.stream()
        .gather(Gatherers.windowFixed(5))
        .forEach(lote -> {
            System.out.println("Nuevo lote");
            lote.forEach(this::esquilar);
        });
```

- En lugar de recibir una oveja por vez, cada elemento del nuevo stream es una lista de hasta cinco ovejas.
- Esto no implica que las cinco se esquilen simultáneamente. Simplemente organiza el trabajo en grupos.



`mapConcurrent()` — limitar el número de esquiladores

```
rebano.stream()
      .gather(Gatherers.mapConcurrent(
          3,
          oveja -> esquilarYObtenerLana(oveja)
      ))
      .forEach(peso ->
          System.out.println("Lana: " + peso + " kg")
      );
```



- En este ejemplo, esquilarYObtenerLana representa una función que esquila una oveja y devuelve los kilogramos de lana obtenidos.

`mapConcurrent(3, ...)` permite tener hasta tres aplicaciones concurrentes de la función, utilizando hilos virtuales. Además, preserva el orden de los resultados del stream.

Diferencia importante: parallelStream() paraleliza el pipeline; mapConcurrent() controla la concurrencia de una operación específica.

`scan()` — calcular la lana acumulada

```
rebano.stream()
.map(Oveja::getKilosLana)
.gather(Gatherers.scan(
        () -> 0,
Integer::sum
    ))
.forEach(System.out::println);
```

A diferencia de una suma tradicional que entrega solamente un resultado final, `scan()` produce resultados acumulados intermedios.



`windowSliding()` — examinar grupos consecutivos

```
rebano.stream()
        .gather(Gatherers.windowSliding(3))
        .forEach(grupo ->
            System.out.println(
                grupo.stream()
                     .map(Oveja::getNombre)
                     .toList()
            )
        );
```

- Esto es útil para analizar secuencias, series temporales o agrupaciones consecutivas.



`windowSliding()` — examinar grupos consecutivos

V1	1	2	3	4	5	6	7	8	9	10
V2	1	2	3	4	5	6	7	8	9	10
V3	1	2	3	4	5	6	7	8	9	10
V4	1	2	3	4	5	6	7	8	9	10
V5	1	2	3	4	5	6	7	8	9	10
V6	1	2	3	4	5	6	7	8	9	10
V7	1	2	3	4	5	6	7	8	9	10
V8	1	2	3	4	5	6	7	8	9	10



`fold()` — obtener un resumen final

```
int total = rebano.stream()
.map(Oveja::getKilosLana)
.gather(Gatherers.fold(
        () -> 0,
        Integer::sum
    ))
    .findFirst()
    .orElse(0);
```



`fold()` emite un resultado acumulado final, a diferencia de `scan()`, que entrega los acumulados sucesivos.

NOTA: `fold` y `scan` se evalúan secuencialmente aunque el stream sea paralelo, mientras que `reduce` con una operación asociativa sí se paraleliza.

Un pipeline completo: filtrar, transformar, reducir

```
List<Integer> numeros = Arrays.asList(1, 2, 3, 4, 5, 6);

 int suma = numeros.stream()
                   .filter(n -> n % 2 == 0)
                   .mapToInt(n -> n * n)
                   .sum();

System.out.println(suma); // Imprime 56 (4 + 16 + 36)
```

n -> n * n es la versión en Java de **λn. n × n** del cálculo lambda: una función anónima que recibe n y retorna su cuadrado. Java no obliga a que una lambda sea pura; eso depende de quien la escribe.

Streams de Java

En lugar de mutar de estado, podemos pensar que los objetos posteriores son una siguiente etapa evolutiva de un objeto anterior sin noción de tiempo.

- Sin streams [pseudocódigo]

```
for (s: students)
   if (s is active)
      active.add(s);
for (a: active)
   agesum = agesum + a.age();
avg = agesum/active.size();
```

- Con streams [pseudocódigo]

```
students.stream().filter(s->s is active).map(s->s.age).average();
```

Piense que es similar al ejemplo de una variable x que abarca los números enteros positivos. Podríamos pensar que x cambia a medida que el tiempo pasa de $t = 1$, $t = 2$, $t = 3$, etc. O podríamos pensar en x como una función que existe de una sola vez. En este caso, nada ha cambiado. Un stream puede considerarse de la misma manera. Es una cuestión de perspectiva. Hay muchas librerías de streaming y varían en diseño y propósito. Veremos algunas en diferentes lenguajes de programación.

Streams en otros lenguajes

- **Varios lenguajes tienen elementos conceptualmente equivalentes** a los *Streams* de Java, aunque no siempre con el mismo nombre ni exactamente el mismo modelo. La idea común es esta: **procesar colecciones o secuencias de datos de forma declarativa**, encadenando operaciones como filtrar, transformar, mapear, reducir o agrupar, en vez de escribir bucles imperativos paso a paso.

- **Python**: se parece con map, filter, reduce, las **generator expressions** y los **iterators/generators**. No es idéntico a Java Streams, pero cumple una función muy similar.
- **Rust**: su equivalente más claro son los **iterators**. De hecho, el estilo iter().map(...).filter(...).collect() es muy cercano a Java Streams.
- **Go**: no tiene un equivalente nativo tan directo como Java Streams. Se trabaja más con bucles, slices y funciones, aunque existen librerías y patrones con **channels** o funciones genéricas que se aproximan.
- **Erlang**: no usa “streams” al estilo Java, pero sí tiene procesamiento funcional sobre listas con funciones como map, filter y fold, sobre todo en el módulo lists.

Streams en otros lenguajes

- En los **lenguajes funcionales**, la equivalencia suele ser incluso más natural que en Java, porque ese estilo es parte central del lenguaje.
- Por ejemplo, **Haskell** usa **listas perezosas** y funciones como map, filter y fold;
- **Clojure** usa **sequences**;
- **F#** usa seq;
- **Scala** tiene colecciones funcionales y también tuvo Stream y luego LazyList; y
- **Elixir** distingue entre Enum y Stream.

- Entonces, la respuesta corta es:
  - **sí, casi todos tienen algo equivalente en propósito, pero a veces se basa en iteradores, secuencias perezosas, generadores o funciones de orden superior, más que en una clase llamada literalmente Stream**.

Determinismo y condiciones de carrera

Determinismo y condiciones de carrera

- **Determinismo funcional**: Siempre, a la misma entrada le corresponde la misma salida.
- **Determinismo estructural**: Siempre, a la misma entrada le corresponde el mismo grafo computacional (la misma estructura del programa en paralelo)
- Problema de Data Race
  - Read-Write
  - Write-Write
  - No hay problema con read-read
- **DRF (*Data Race Freedom*):** ningún par de accesos que puedan ejecutarse en paralelo toca la misma posición de memoria con al menos una escritura. Si un programa con async/finish y futuros es DRF, entonces es determinista funcional y estructuralmente.

Una condición de carrera en el rebaño

Con carrera

```
int[] total = {0};
rebano.parallelStream()
      .forEach(o ->
          total[0] += o.getKilosLana());
```

Varios hilos leen y escriben `total[0]` sin orden garantizado: hay conflictos escritura-escritura y lectura-escritura.
Esperado: 3 000 000. Obtenido: puede ser menor y cambiar en cada ejecución.

Libre de carreras (DRF)

```
int total = rebano.parallelStream()
      .mapToInt(Oveja::getKilosLana)
      .sum();
```

Cada hilo suma su parte y la librería combina los resultados parciales: no hay estado compartido mutable.
Siempre 3 000 000: determinismo funcional.

Las dos versiones suman la misma lana, pero la de la izquierda deja que varios hilos escriban sobre la misma variable sin orden y pierde actualizaciones, mientras que la de la derecha no comparte estado mutable y por eso da siempre 3 000 000: así se ve en la práctica que la ausencia de data races garantiza el determinismo.

NO-Determinismo benigno

- Con la misma entrada se obtienen salidas diferentes, pero son resultados aceptables [todas son correctas según la especificación]
  - Ejemplo, diferentes búsquedas web en el mismo buscador, para las mismas palabras con “diferentes” resultados, pero todos aceptables

```
Optional<Oveja> alguna = rebano.parallelStream()
        .filter(Oveja::necesitaEsquila)
              .findAny();
```

Cada ejecución puede devolver una oveja distinta: todas necesitan esquila, así que cualquiera es válida.

- Diferencia con una condición de carrera: aquí la especificación admite varias respuestas; en la carrera, el resultado es simplemente incorrecto.

En resumen: paralelismo sin carreras

Funciones puras

Mismo argumento, mismo resultado; sin efectos secundarios

Futuros

Expresan dependencias; join() y get() son los puntos de sincronización

Memoización

Reutiliza resultados; con futuros, también el trabajo en curso

Streams

Qué hacer, no cómo; secuencial, paralelo o concurrente

Determinismo

Sin data races: misma salida y mismo grafo

Idea central: si no hay estado compartido mutable, ejecutar en paralelo no cambia el resultado.

Gracias